

Softwarekonstruktion WS 15/16 – Übung 6

Ausgabe: 18.01.2016, Abgabe: 01.02.2016 12:00 Uhr

Präsenzübung

6.1 Signatur und Algebra

Gegeben Sei folgende Signatur:

```
 $\Sigma =$  sorts  int  
      opns  zero:  $\rightarrow$  int  
            succ: int  $\rightarrow$  int  
            inv: int  $\rightarrow$  int
```

Geben Sie eine Σ -Algebra A an, so dass **zero**, **succ** und **inv** die generierenden Operationen der Menge der ganzen Zahlen sind.

Hinweis: Die Operation **inv** soll einen Vorzeichenwechsel für alle Zahlen außer 0 bewirken.

Lösungsvorschlag

$$\begin{aligned} A_{\text{int}} &:= \mathbb{Z} \\ \text{zero}_A &:= 0 \\ \text{succ}_A(z) &:= 1 + z, \forall z \in \mathbb{Z} \\ \text{inv}_A(z) &:= -z, \forall z \in \mathbb{Z} \setminus 0 \end{aligned}$$

6.2 Algebraische Spezifikation: Liste

Erstellen Sie die algebraische Spezifikation einer Liste mit folgenden Operationen:

1. **new**: Erzeugt eine leere Liste
2. **add**: Fügt ein Element an das Ende der Liste an
3. **del**: Löscht ein Element am Ende der Liste
4. **con**: Fügt zwei Listen aneinander
5. **head**: Liefert das erste Element der Liste
6. **tail**: Liefert das letzte Element der Liste
7. **len**: Liefert die Länge der Liste

Die Elemente der Liste sind vom abstrakten Datentyp **elem**.

Lösungsvorschlag

$SPEC_{List} =$

- **sorts**

- list, elem, nat

- **opns**

- zero: $\rightarrow$ nat
- succ: nat $\rightarrow$ nat
- new: $\rightarrow$ list
- add: list $\times$ elem $\rightarrow$ list
- del: list $\rightarrow$ list
- con: list $\times$ list $\rightarrow$ list
- head: list $\rightarrow$ elem
- tail: list $\rightarrow$ elem
- len: list $\rightarrow$ nat

- **vars**

- L_1, L_2 : list
- e : elem

- **eqns**

- (head(new())) = n.d. // f. new
- (tail(new())) = n.d. // f. new
- len(new()) = zero() // f. new
- head(add(L_1 , e)) = head(L_1) // f. add
- head(add(new()), e) = e // f. add
- tail(add(L_1 , e)) = e // f. add
- len(add(L_1 , e)) = succ(len(L_1)) // f. add
- del(new()) = new() // f. del
- del(add(L_1 , e)) = L_1 // f. del
- con(L_1 , new()) = L_1 // f. con
- con(L_1 , add(L_2 , e)) = add(con(L_1 , L_2), e) // f. con

Hausübung

6.3 Signatur und Algebra: Reelle Zahlen (2 Punkte)

Gegeben sei folgende Signatur der reellen Zahlen:

```
Σ = sorts  real
      opns  zero: → real
            one: → real
            add: real × real → real
            sub: real × real → real
            mult: real × real → real
            div: real × real → real
```

Geben Sie eine Algebra an, die die Signatur interpretiert.

Lösungsvorschlag

$$\begin{aligned}
 A_{\text{real}} &:= \mathbb{R} \\
 \text{zero}_A &:= 0 \\
 \text{one}_A &:= 1 \\
 \text{add}_A(x, y) &:= x + y & \forall x, y \in \mathbb{R} \\
 \text{sub}_A(x, y) &:= x - y & \forall x, y \in \mathbb{R} \\
 \text{mult}_A(x, y) &:= x \cdot y & \forall x, y \in \mathbb{R} \\
 \text{div}_A(x, y) &:= \frac{x}{y} & \forall x, y \in \mathbb{R}, y \neq 0
 \end{aligned}$$

Dies ist nur eine von vielen interpretierenden Algebren. Prinzipiell ginge als Minimalbeispiel z.B. auch:

$$\begin{aligned}
 B_{\text{real}} &:= \{\phi\} \\
 \text{zero}_B &:= \phi \\
 \text{one}_B &:= \phi \\
 \text{add}_B(x, y) &:= \phi \quad \forall x, y \in \{\phi\} \\
 \text{sub}_B(x, y) &:= \phi \quad \forall x, y \in \{\phi\} \\
 \text{mult}_B(x, y) &:= \phi \quad \forall x, y \in \{\phi\} \\
 \text{div}_B(x, y) &:= \phi \quad \forall x, y \in \{\phi\}
 \end{aligned}$$

6.4 Algebraische Spezifikation: Stack (5 Punkte)

Geben Sie eine algebraische Spezifikation für eine Stack-Datenstruktur mit den folgenden Operationen an:

1. `empty` : erzeugt einen leeren Stack
2. `push` : legt ein Element auf den Stack
3. `top` : gibt das oberste Element des Stacks zurück, sofern es eines gibt, sonst 0
4. `pop` : entfernt das oberste Element des Stacks, sofern es eines gibt
5. `size` : liefert die Anzahl der Elemente im Stack

Lösungsvorschlag

```

SPECstack = sorts  stack, elem, nat
               opns  empty: → stack
                   push: stack × elem → stack
                   top: stack → elem ∪ nat
                   pop: stack → stack
                   size: stack → nat
                   0: → nat
                   succ: nat → nat
               vars  s: stack
                   e: elem
               eqns  size(empty()) = 0
                   size(push(s,e)) = succ(size(s))
                   top(push(s,e)) = e
                   top(empty()) = 0
                   pop(push(s,e)) = s
                   pop(empty()) = empty()

```

6.5 Algebraische Spezifikation: Eigenschaften von Algebren (4 Punkte)

Vervollständigen Sie:

1. Eine algebraische Spezifikation ist *redundant*, wenn ...
2. Eine algebraische Spezifikation ist *widersprüchlich*, wenn ...
3. Eine Algebra folgt bzgl. einer algebraischen Spezifikation dem Prinzip *no-junk*, wenn ...
4. Eine Algebra folgt bzgl. einer algebraischen Spezifikation dem Prinzip *no-confusion*, wenn ...

Lösungsvorschlag

1. Eine algebraische Spezifikation ist *redundant*, wenn sich ein Axiom entfernen lässt und die Menge der beweisbaren Aussagen gleich bleibt.
2. Eine algebraische Spezifikation ist *widersprüchlich*, wenn sich beweisen lässt, dass zwei als unterschiedlich definierte Elemente gleich sind.
3. Eine Algebra folgt bzgl. einer algebraischen Spezifikation dem Prinzip *no-junk*, wenn sie nur Elemente enthält, die durch Terme der Signatur bezeichnet werden können.

4. Eine Algebra folgt bzgl. einer algebraischen Spezifikation dem Prinzip *no-confusion*, wenn nur diejenigen Eigenschaften gelten, die durch die Spezifikation ausgedrückt werden.

6.6 Algebraische Spezifikation: Auswertung I (4 Punkte)

Gegeben sei die Spezifikation einer Liste, die insbesondere folgende Axiome enthält.

$$\text{I } \text{head}(\text{add}(L, e)) = \text{head}(L)$$

$$\text{II } \text{head}(\text{add}(\text{new}(), e)) = e$$

Zeigen Sie, wie sich mit den beiden Axiomen der Funktionswert $\text{head}([1, 2, 3, 4])$ berechnen lässt.

Annahme: die Liste $[1, 2, 3, 4]$ entsteht durch $\text{add}([1, 2, 3], 4)$.

$$\begin{aligned} \text{head}([1, 2, 3, 4]) &= \text{head}(\text{add}([1, 2, 3], 4)) && \text{wegen (I) folgt} \\ &= \text{head}([1, 2, 3]) \\ &= \text{head}(\text{add}([1, 2], 3)) && \text{wegen (I) folgt} \\ &= \text{head}([1, 2]) \\ &= \text{head}(\text{add}([1], 2)) && \text{wegen (I) folgt} \\ &= \text{head}([1]) \\ &= \text{head}(\text{add}(\text{new}(), 1)) && \text{wegen (II) folgt} \\ &= 1 \end{aligned}$$

6.7 Algebraische Spezifikation: Auswertung II (5 Punkte)

Gegeben sei die Spezifikation einer Liste, die insbesondere folgende Axiome enthält.

$$\text{I } \text{con}(L, \text{new}()) = L$$

$$\text{II } \text{con}(L_1, \text{add}(L_2, e)) = \text{add}(\text{con}(L_1, L_2), e)$$

Zeigen Sie, dass sich die Gleichung (III) $\text{con}(\text{new}(), L) = L$ aus den Axiomen ableiten lässt.

Es wird davon ausgegangen, dass $\text{new}()$ eine neue, leere Liste erzeugt und dass $\text{add}(L, e)$ einer beliebigen Liste L ein Element e hinzufügt und in jedem Fall eine nichtleere Liste zurückgibt.

- Für *leere Listen* L gilt $L = \text{new}()$ und damit

$$\begin{aligned} \text{con}(\text{new}(), L) &= \text{con}(\text{new}(), \text{new}()) && \text{wegen (I) folgt} \\ &= \text{new}() = L \end{aligned}$$

- Für *nichtleere Listen* L wird der Beweis durch strukturelle Induktion erbracht:

- Induktionsanfang: Für Listen L der Länge 1 gilt $L = \text{add}(\text{new}(), e)$ und damit

$$\begin{aligned}\text{con}(\text{new}(), L) &= \text{con}(\text{new}(), \text{add}(\text{new}(), e)) && \text{wegen (II) folgt} \\ &= \text{add}(\text{con}(\text{new}(), \text{new}()), e) && \text{wegen (I) folgt} \\ &= \text{add}(\text{new}(), e) = L\end{aligned}$$

- Induktionsvoraussetzung: Gleichung (III) gelte für Listen L beliebiger Länge n .
- Induktionsschritt: Für Listen L' mit Länge $n+1$ gilt, dass sie sich als $L' = \text{add}(L, e)$ darstellen lassen und somit:

$$\begin{aligned}\text{con}(\text{new}(), L') &= \text{con}(\text{new}(), \text{add}(L, e)) && \text{wegen (II) folgt} \\ &= \text{add}(\text{con}(\text{new}(), L), e) \\ &\stackrel{IV}{=} \text{add}(L, e) = L'\end{aligned}$$

Gleichung (III) gilt also für Listen beliebiger Länge.

6.8 Kopplung und Kohäsion: Metrik (4 Punkte)

Überlegen Sie sich Metriken zur Messung von Kopplung und Kohäsion eines Software-Systems. Beschreiben Sie diese und veranschaulichen Sie sie an einem kleinen Beispiel.

6.9 Architekturstile: Beispiele (8 Punkte)

Beschreiben Sie für drei der folgenden Architekturstile Beispiele. Sie können auch ein System beschreiben, das mehrere Stile vereint.

1. Client-Server-Architektur
2. Objekt-Broker
3. Peer-to-peer-System
4. Ereignisgesteuertes System
5. Publish-subscribe
6. Schichten-Architektur
7. Repositories
8. Blackboard-System
9. Model-View-Controller

6.10 Entwurfsprinzipien: Beispiele (8 Punkte)

Beschreiben Sie für drei der folgenden Entwurfsprinzipien jeweils ein Beispiel:

1. Separation of Concerns

2. Single Responsibility Principle
3. Open-Closed Principle
4. Interface-Segregation Principle
5. Dependency Inversion Principle
6. Law of Demeter
7. Design by Contract
8. Don't Repeat Yourself
9. Self-documentation
10. Design for change